



NATIONAL UNIVERSITY OF SCIENCES AND TECHNOLOGY
DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING
COLLEGE OF E&ME, NUST, RAWALPINDI

EC-200 — Data Structures

PROJECT REPORT

Adaptive Traffic Flow Optimization System

Using Graph Theory, Dijkstra's Algorithm & Adaptive Signal Control

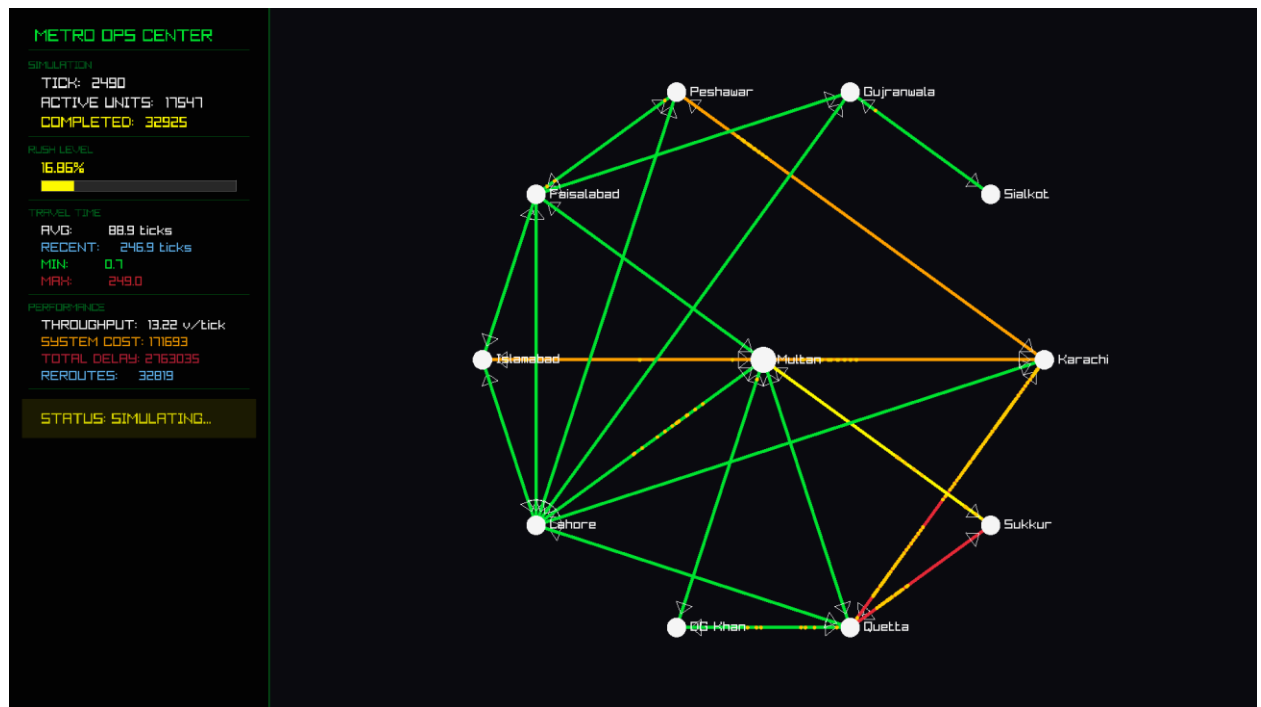
GROUP MEMBERS

Student Name	CMS Roll No.
Mohammad Usman Irshad	533076
Abdullah Aqeel Khan	501845
Muhammad Husnain	507214

Instructor	Dr. Anum Abdul Salam
Lab Engineer (LE)	Hira Irshad
Course	EC-200 — Data Structures
Submission Date	May 03, 2026
Difficulty Level	Complex Engineering Problem (CEP)
Language / Tools	C++17 · Visual Studio 2022 · Raylib 5.0 · Windows 11

Table of Contents

Table of Contents.....	2
1. Introduction	5
1.1 Background of the Application Domain	5
1.2 Brief System Overview	5
2. Problem Statement	5
2.1 Clear Definition of the Problem.....	5
2.2 Limitations of Existing Approaches	6
3. Objectives	6
3.1 Primary Objective	6
3.2 Secondary Objectives	6
3.3 System Overview	7
3.4 System Architecture Diagram	7
4. System Design	8
4.1 Modular Class Architecture	8
4.2 Inter-Module Communication	9
4.3 Key Design Decisions	9
5. Mathematical Modelling	9
5.1 Road Network Model.....	9
5.2 Traffic Flow Update	9
5.3 BPR Travel Time Model	10
5.4 Shortest-Path Routing (Dijkstra with Priority Queue).....	10
5.5 Adaptive Traffic Signal Model.....	10
5.6 Global Objective Function	10
6. Implementation Details	10
6.1 Development Environment	10
6.2 Project File Structure	10
6.3 Key Implementation Decisions	11
7. Key Features	11
8. Results and Outputs	12
8.1 Raylib GUI Visualization	12



.....	17
8.2 Performance Metrics Output	18
9. Time & Space Complexity Analysis	19
9.1 Per-Function Complexity.....	19
9.2 Per-Tick Breakdown	20
9.3 Overall Simulation Complexity	20
9.4 Scalability Analysis	21
9.5 Space Complexity Summary.....	21
9.6 Summary	21
10. Challenges and Solutions	22
10.1 Graph Construction and Directed Edge Representation.....	22
10.2 Consistent Tick-Order Updates.....	22
10.3 Stale Entries in Dijkstra’s Priority Queue	22
10.4 Raylib Rendering Synchronization	22
10.5 Scalable Vehicle Management.....	22
12. Entire Project Complexity — End-to-End Analysis	22
12.1 Simulation Lifecycle Phases	22
12.2 Complete Time Complexity Formula	23
12.3 Concrete Breakdown for This Project	23

12.4 Complete Space Complexity	24
12.5 Complexity Class of the Entire System	24
13. Conclusion	25
References	25

1. Introduction

1.1 Background of the Application Domain

Modern urban environments are increasingly challenged by severe traffic congestion driven by rising vehicle density and the limitations of traditional, static traffic control infrastructure. Fixed-cycle traffic signals and pre-planned routing systems are inherently unable to adapt to the dynamic, real-time fluctuations that characterize actual road networks. The direct consequences include increased travel times, higher fuel consumption, greater emissions, and degraded urban productivity.

Graph theory provides the mathematical backbone for modeling road networks as weighted directed graphs, where intersections become nodes and road segments become edges carrying capacity and flow attributes. The discipline of algorithmic optimization — particularly shortest-path computation via Dijkstra's algorithm — gives rise to systems that can compute, in real time, the least-cost path for any vehicle given current congestion levels. When coupled with queue-based intersection models and adaptive signal control strategies, the result is a simulation framework that closely approximates real-world intelligent transportation systems.

1.2 Brief System Overview

The Adaptive Traffic Flow Optimization System models Pakistan's inter-city road network as a directed graph and simulates multi-vehicle traffic over discrete time steps. At each simulation tick, vehicle flows are updated, road congestion and travel times are recalculated using the Bureau of Public Roads (BPR) formula, optimal routes are recomputed via Dijkstra's algorithm (optimized with a priority queue min-heap), and traffic signals are adaptively adjusted by granting green time to the intersection approach with the highest cost. The system processes up to 50,000+ vehicles across an 11-city network, implemented in C++ and rendered in real time using the Raylib graphics library.

2. Problem Statement

2.1 Clear Definition of the Problem

Urban road networks exhibit time-varying, non-linear traffic behavior that cannot be managed effectively by static control policies. The core problem is: given a directed road network with defined capacities and speed limits, a stream of vehicles with distinct sources and destinations, and an objective to minimize total network congestion and travel delay, how can a software system dynamically compute optimal vehicle routes and adaptively control intersection signals to achieve near-optimal throughput?

Formally, the network is represented as $G = (V, E)$, where V is the set of intersections and E is the set of directed road segments. Each edge e_{ij} carries parameters including road length

l_{ij} , maximum speed v_{ij_max} , capacity c_{ij} , current flow $f_{ij}(t)$, downstream queue $Q_{ij}(t)$, and congestion-adjusted travel time $w_{ij}(t)$. The system must simultaneously route N vehicles across K source-destination pairs while minimizing the global objective function that balances queue lengths and congestion ratios.

2.2 Limitations of Existing Approaches

- **Static traffic signals** use fixed phase durations regardless of real-time queue lengths, causing unnecessary delay on low-traffic approaches while congested approaches remain blocked.
 - **Pre-planned routing** assigns vehicles to fixed paths computed at departure and does not react to en-route congestion, systematically overloading initially uncongested roads.
 - **Simple shortest-path implementations** ignore flow dynamics and compute routes based on free-flow travel times rather than current traffic states.
 - **Existing simulators** either oversimplify the traffic model (ignoring queuing and capacity constraints) or are too computationally expensive for real-time interactive simulation.
-

3. Objectives

3.1 Primary Objective

To design and implement a fully functional, modular C++ simulation of an urban traffic network that combines graph-theoretic road modeling, congestion-aware Dijkstra routing, queue-based intersection dynamics, and adaptive traffic signal control — fulfilling the Complex Engineering Problem requirements of the EC-200 Data Structures course.

3.2 Secondary Objectives

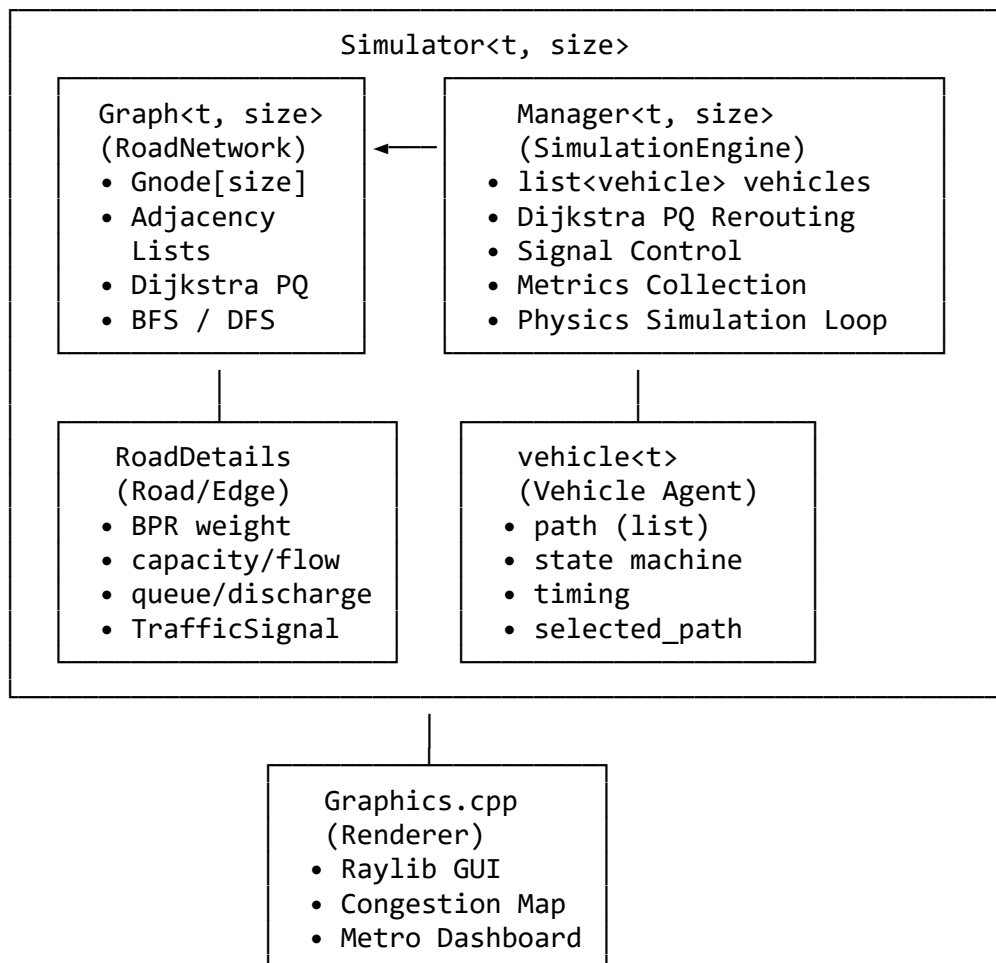
1. Implement a directed weighted graph class (`Graph<t, size>`) to represent the road network with adjacency-list storage.
2. Implement the BPR travel-time model (`RoadDetails`) to dynamically update edge weights based on real-time congestion ratios.
3. Implement Dijkstra's algorithm with a priority queue min-heap to compute optimal vehicle routes in $O((V + E) \log V)$ time.
4. Implement a queue-based intersection model that tracks $Q_{ij}(t)$ per incoming road and discharges vehicles based on signal state, discharge rate, and downstream capacity.
5. Implement an adaptive traffic signal controller (`TrafficSignal`) granting green based on a cost function combining queue length, congestion, and starvation cost.
6. Implement a Vehicle class (`vehicle<t>`) modeling per-vehicle state with a 4-state machine (uninitialized, queued, travelling, arrived).

7. Implement a performance metrics module tracking average travel time, total delay, throughput, P95 travel time, reroute count, and average network congestion.
8. Develop a real-time Raylib-based graphical interface rendering the road network, vehicle positions, signal states, and a congestion heat map.

3.3 System Overview

The simulation operates as a discrete-time loop. Each tick executes ordered phases: (1) signal state updates, (2) vehicle time decrements, (3) arrival handling, (4) intersection transitions with Dijkstra rerouting, (5) queue-to-edge transitions, (6) entrance processing, (7) broken path cleanup, and (8) metric accumulation. This architecture mirrors the real-time control loop of deployed intelligent transport systems while remaining computationally tractable for a single-machine C++ implementation.

3.4 System Architecture Diagram



4. System Design

4.1 Modular Class Architecture

Class / Module	Responsibility	Key Data Structure	Design Approach
Graph<t, size> (RoadNetwork)	Models the city as directed graph $G=(V,E)$. Stores all intersections and roads.	Fixed-size array <code>Gnode<t>[size]</code> with <code>list<weighted></code> adjacency lists	Each node = city vertex; each edge = Road with capacity, flow, queue, and BPR params
RoadDetails (Road/Edge)	Directed road segment e_{ij} . Holds length, max_speed, capacity, currentVehicles, queueCount, signalState.	Plain struct with float/int fields + embedded <code>TrafficSignal</code>	BPR travel time computed on demand: $w = w_{\text{free}} \times (1 + \alpha \times (f/c)^\beta)$
TrafficSignal	Manages per-road signal timing with starvation-aware cost-based switching.	float timers (green/red), cost params (pa, pb)	Adaptive cost-priority policy with min/max green duration and bottleneck protection
vehicle<t>	Independent agent: source, destination, current road, remaining travel time, status.	<code>list<t></code> path, <code>vector<t></code> selected_path, <code>pair<t,t></code> currentRoad	4-state machine: -1 (uninitialized), 0 (queued), 1 (travelling), 2 (arrived)
Manager<t, size> (SimulationEngine)	Master discrete-time loop. Coordinates all update phases each tick.	<code>list<vehicle<t>></code> vehicle pool; reference to Graph	Multi-phase tick: signals → time update → arrivals → routing → queue transitions → metrics
Simulator<t, size>	Network configuration and traffic injection.	Owns Graph* and Manager*	Sets up 11-city Pakistani network; injects 50K randomized vehicles

Class / Module	Responsibility	Key Data Structure	Design Approach
Graphics.cpp (Renderer)	Raylib rendering; draws graph, vehicles, signals, congestion heat map, dashboard.	map<string, Vector2> city positions	Full-screen metro dashboard with real-time metrics sidebar

4.2 Inter-Module Communication

The Manager holds a direct pointer to the Graph and owns the vehicle list. Each simulation tick, it calls `updateSignals()` on all intersections, then processes vehicle time decrements, then `reached()` for arrivals, then `arrivalAtIntersection()` which triggers `ShortestPath()` (Dijkstra rerouting) for vehicles completing a segment, then queue transitions via `entraingfromQueetoEdge()` and `entrance()`. The flow is strictly sequential within each tick to ensure data consistency.

4.3 Key Design Decisions

- **Adjacency list over adjacency matrix:** Sparse road graphs (low average degree) make list storage significantly more memory-efficient and faster to traverse.
- **Priority Queue Dijkstra with lazy deletion:** Stale heap entries from dynamic weight changes are skipped via `if (visited[node]) continue`, avoiding the complexity of decrease-key operations.
- **Cost-based signal policy with starvation prevention:** The cost function $p_a \times \text{queue} + p_b \times \text{congestion}^2 + \text{redTimer} \times 3.5$ balances queue priority with starvation prevention, and bottleneck detection prevents switching away from heavily loaded green roads.
- **BPR travel time:** The Bureau of Public Roads nonlinear formula ($\alpha = 0.1\text{--}0.2$, $\beta = 3.0\text{--}5.0$) provides a well-validated congestion response curve used in real traffic engineering.
- **Sinusoidal traffic injection:** $\text{carsToAdd} = \text{amplitude} \times (\sin(\text{freq} \times t) + 1) + \text{baseline}$ models realistic rush-hour demand surges.

5. Mathematical Modelling

5.1 Road Network Model

$G = (V, E)$, where $V = \{\text{Karachi, Sukkur, Quetta, DG Khan, Multan, Lahore, Islamabad, Faisalabad, Peshawar, Gujranwala, Sialkot}\}$ and $E \approx 100$ directed road edges with multiple parallel routes between major city pairs.

5.2 Traffic Flow Update

$$f_{ij}(t+1) = f_{ij}(t) + a_{ij}(t) - x_{ij}(t)$$

$$Q_{ij}(t+1) = Q_{ij}(t) + x_{ij}(t) - d_{ij}(t)$$

$$d_{ij}(t) = g_{ij}(t) \times \min(Q_{ij}(t), \mu_{ij}, c_{jk} - f_{jk}(t))$$

5.3 BPR Travel Time Model

$$w_{ij}(t) = w_{ij_free} \times (1 + \alpha \times (f_{ij}(t) / c_{ij})^\beta)$$

$$w_{ij_free} = l_{ij} / v_{ij_max}$$

5.4 Shortest-Path Routing (Dijkstra with Priority Queue)

$$P_v = \operatorname{argmin} \sum w_{ij}(t) \text{ along path from } s_v \text{ to } d_v$$

Implemented with `std::priority_queue<pair<float,int>, vector<...>, greater<...>>` for $O(\log V)$ extract-min.

5.5 Adaptive Traffic Signal Model

$$\text{cost}(\text{road}) = p_a \times Q_{ij}(t) + p_b \times (f_{ij}/c_{ij})^2 + \text{starvationCost}$$

$$\text{starvationCost} = \text{redTimer} \times 3.5$$

Green is granted to the road with maximum cost. Switching is blocked if `greenTimer < minGreenTime` and forced if `greenTimer ≥ maxGreenTime`.

5.6 Global Objective Function

$$\min \sum_t \sum(i,j) [\alpha \times Q_{ij}(t) + \beta \times (f_{ij}(t)/c_{ij})^2]$$

6. Implementation Details

6.1 Development Environment

Language	C++17
IDE	Visual Studio 2022
Graphics	Raylib 5.0 (statically linked via NuGet)
Platform	Windows 11

6.2 Project File Structure

File	Role
Graphics.cpp	Entry point; initializes Simulator and Raylib window, runs the simulation/render loop
Graph.h	Graph<t, size> class — adjacency-list graph, Dijkstra PQ, BFS, DFS, Prim's MST

File	Role
RoadDetails.h	Road edge struct — BPR travel time, congestion ratio, vehicle/queue management
TrafficSignal.h	Signal controller — green/red timers, starvation cost, canSwitch() policy
vehicle.h	Vehicle struct — 4-state machine, path tracking, timing
Manger.h	Manager<t, size> — simulation engine, signal control, metrics, physics loop
Simulator.h	Network setup (11 cities, ~50 directed edges), traffic injection (50K vehicles)
Header1.h	Unified include header

6.3 Key Implementation Decisions

- The priority queue in Dijkstra uses `std::priority_queue<pair<float,int>, vector<...>, greater<...>>` for $O(\log V)$ extract-min.
- Vehicles are re-routed via Dijkstra at each tick through `ShortestPath()` to react to real-time congestion changes.
- The simulation tick rate is configurable; Raylib's `SetTargetFPS(60)` synchronizes rendering to the simulation speed.
- Sinusoidal traffic injection (`injectSinusoidalTraffic()`) adds 5–25 vehicles every 50 ticks to model demand surges.

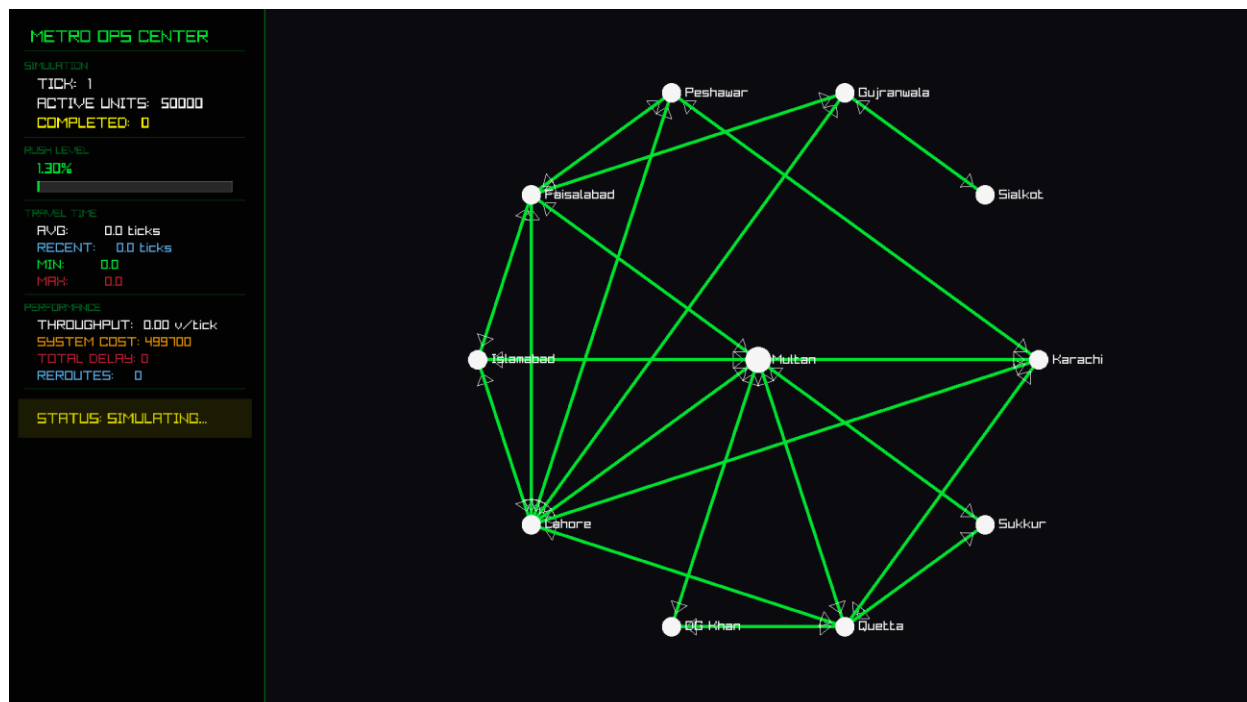
7. Key Features

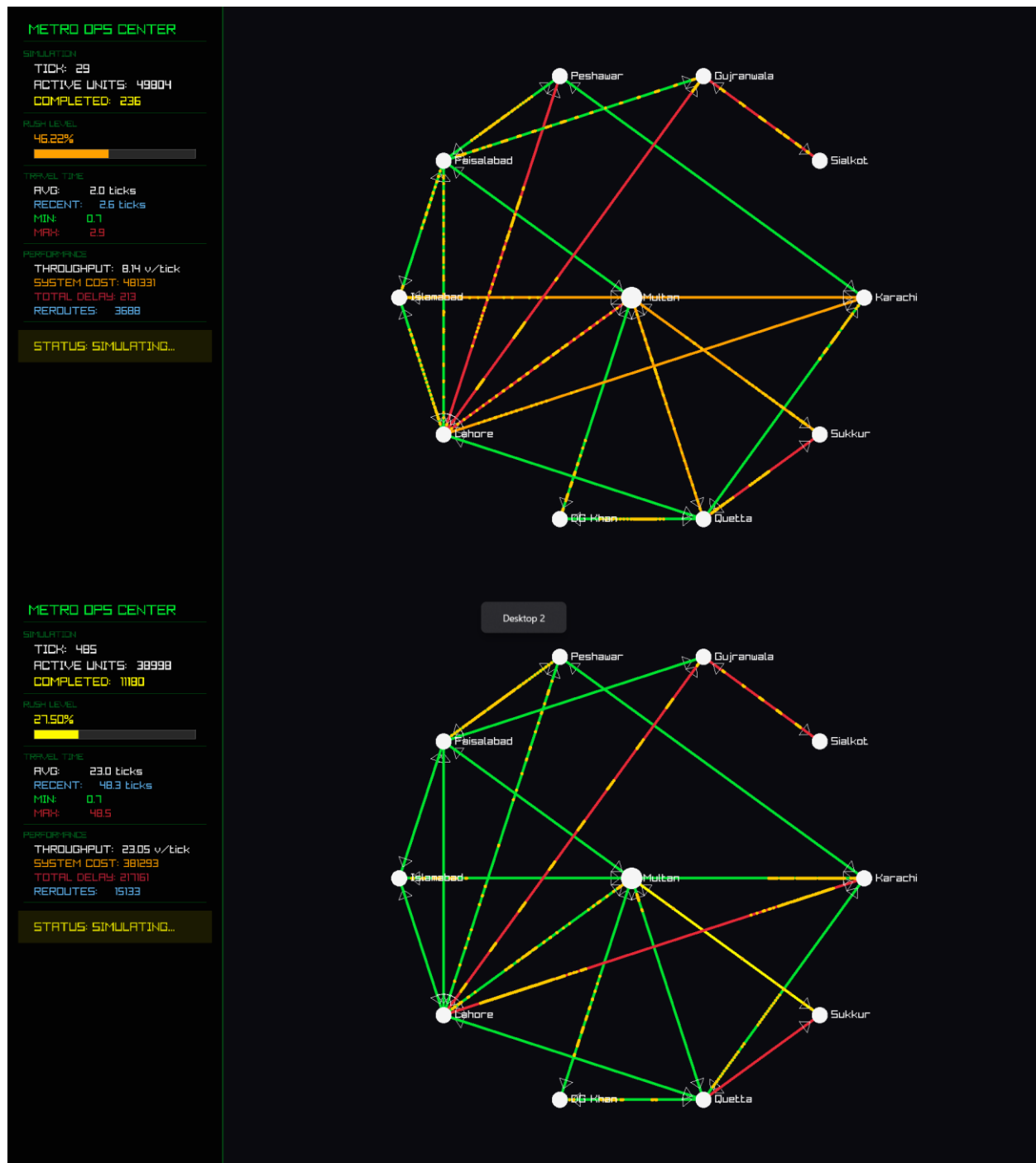
1. **Dynamic Directed Graph:** 11-city Pakistani road network with ~100 directed edges, multiple parallel routes, and hub-spoke topology centered on Multan.
2. **BPR Congestion-Aware Routing:** Travel times update every tick via the BPR formula; Dijkstra routes vehicles around congestion automatically.
3. **Adaptive Signal Control:** Intersection signals respond to live queue measurements and congestion ratios with starvation prevention — no fixed cycle lengths.
4. **Queue-Based Discharge Model:** Vehicles obey signal state, discharge rate (`DischargeAllowed()`), and downstream capacity constraints before advancing.
5. **Massive Scale:** 50,000+ vehicles with independent source-destination pairs simulated simultaneously with sinusoidal injection waves.
6. **Real-Time Raylib Visualization:** Full-screen metro dashboard with congestion heatmap, vehicle animation, rush-level bar, and live metrics.
7. **Comprehensive Metrics:** Average/windowed/min/max/P95 travel time, throughput, total delay, system cost, reroute count — logged to CSV and text files.

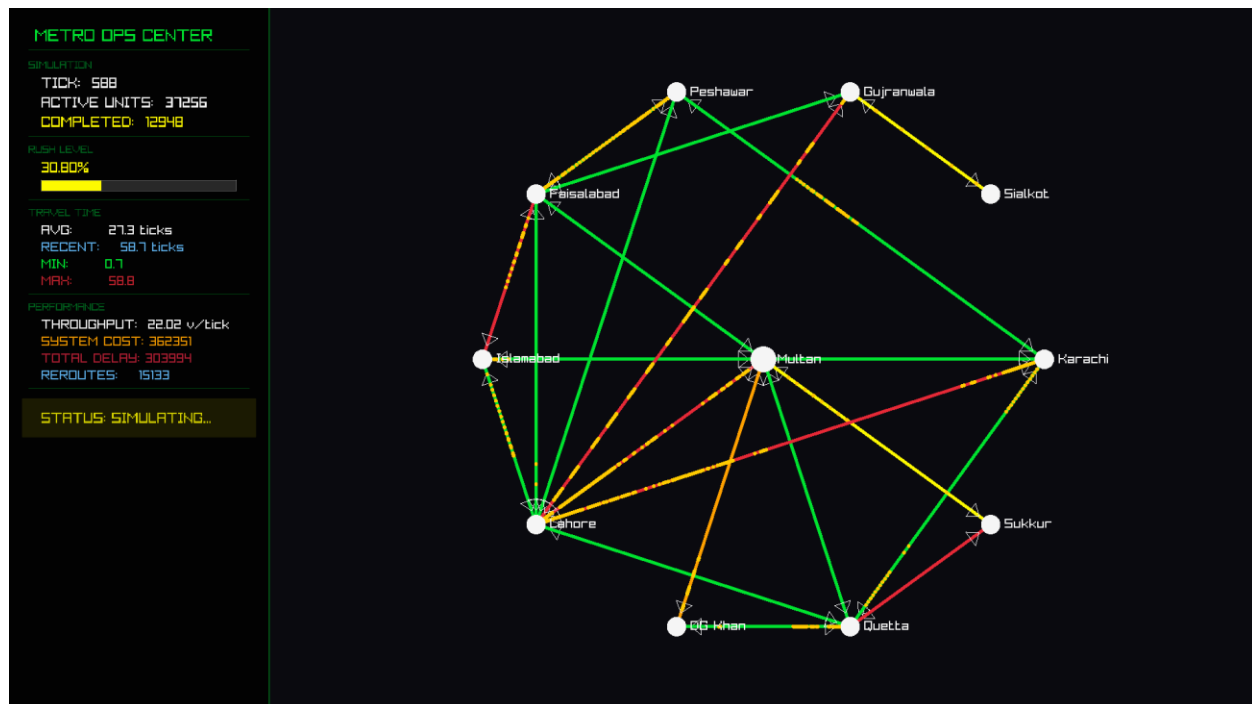
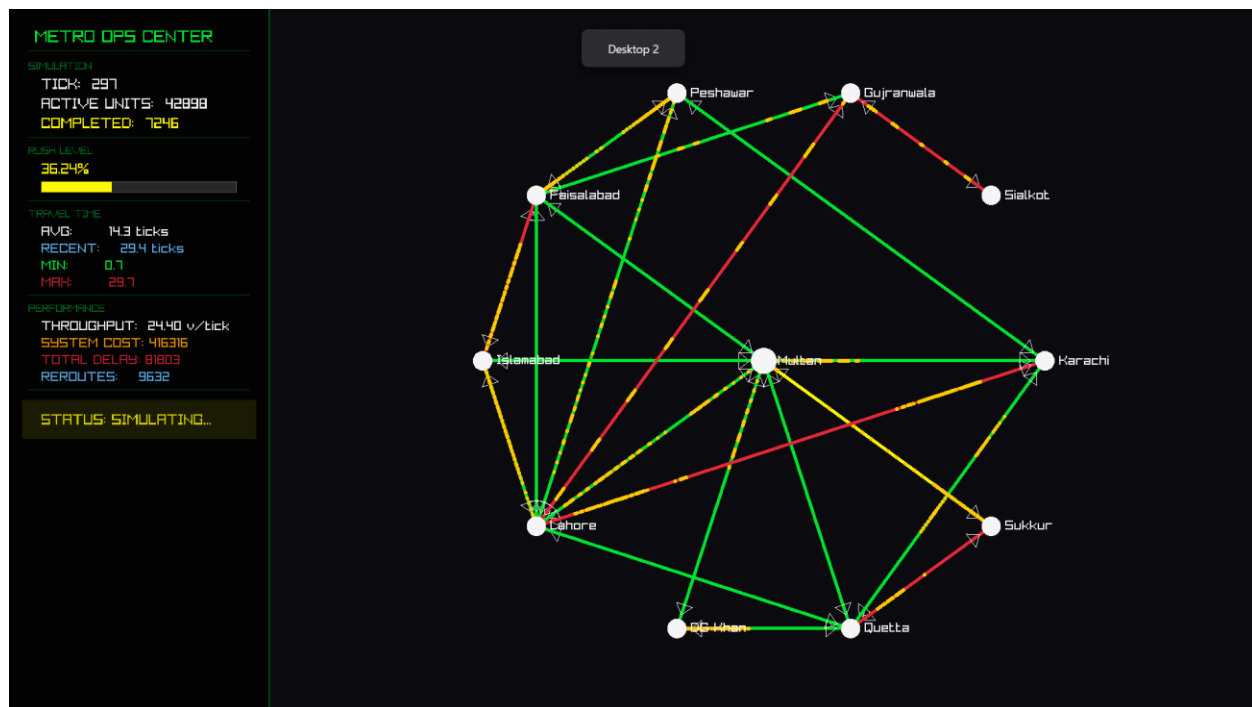
8. Results and Outputs

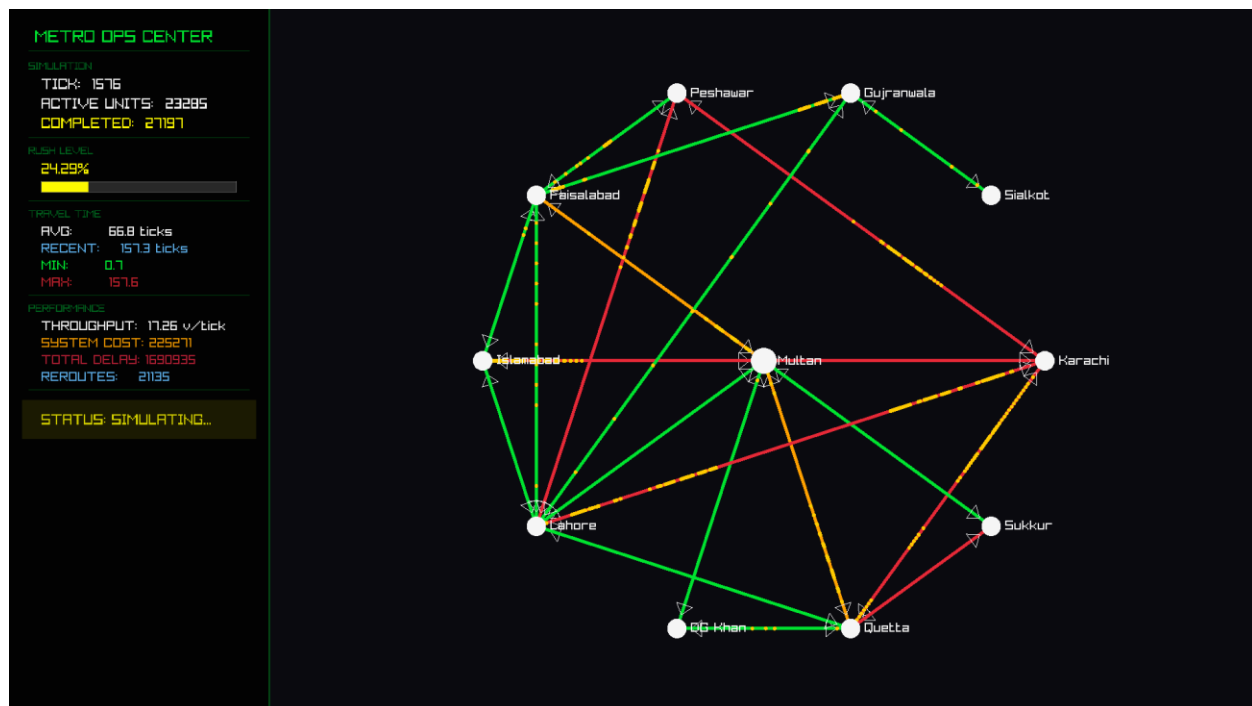
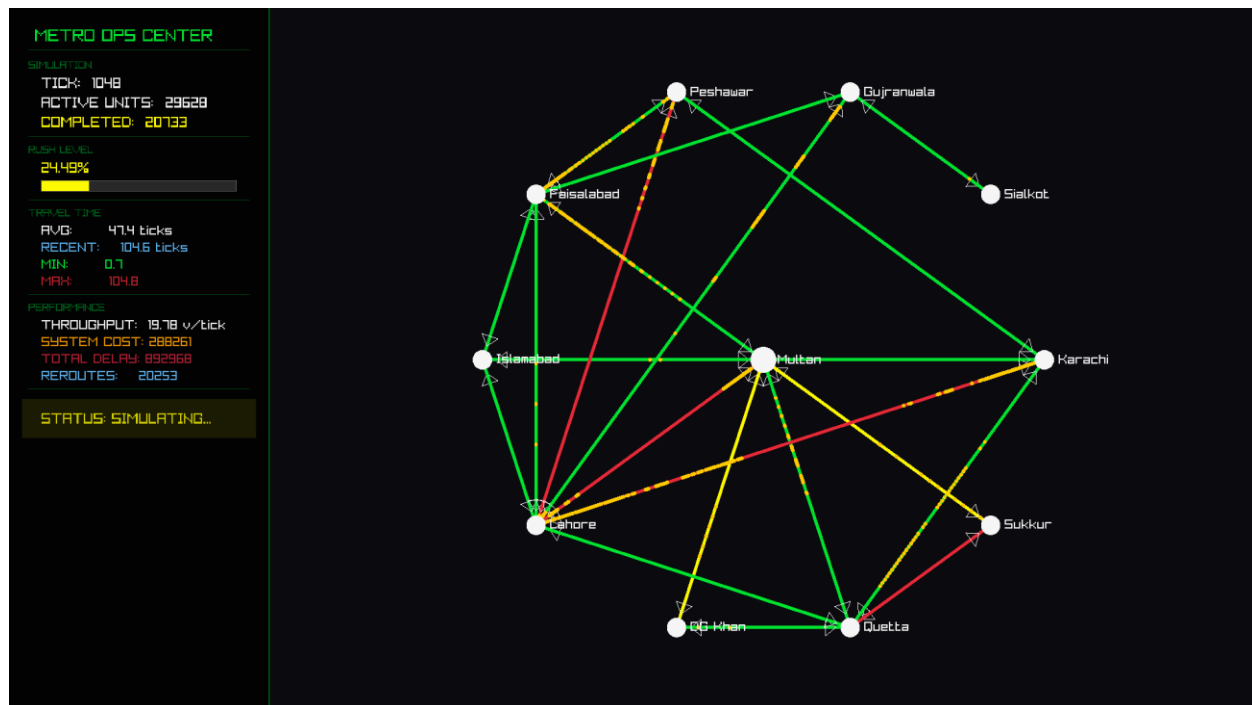
8.1 Raylib GUI Visualization

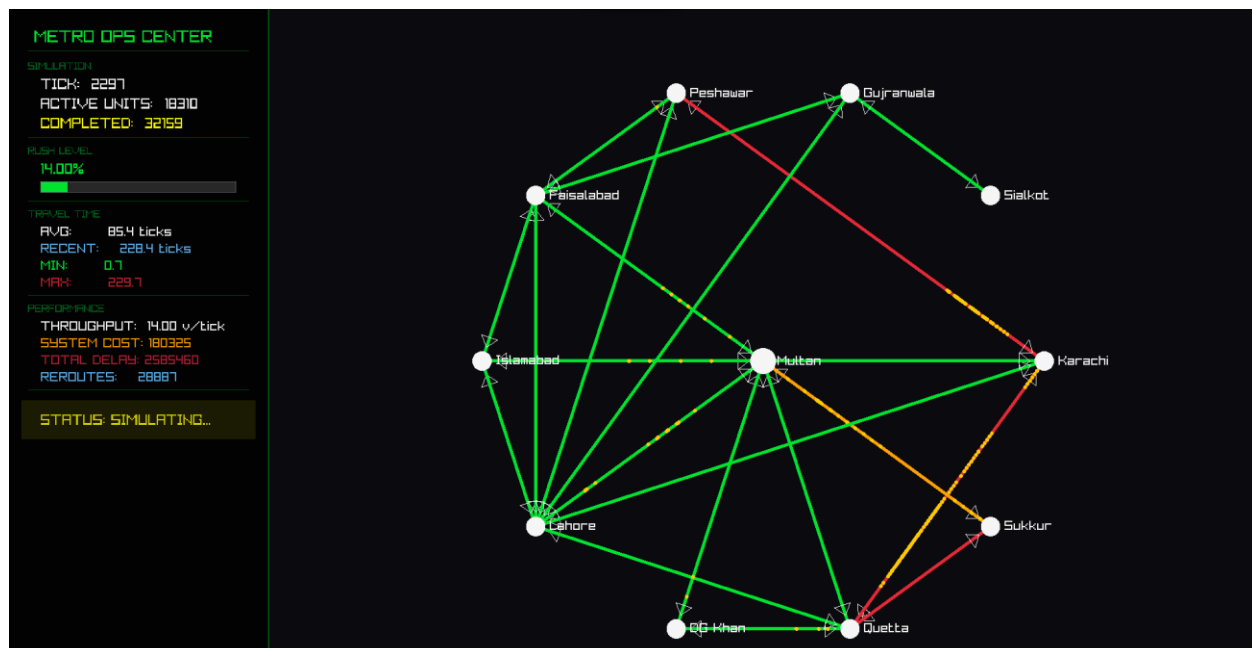
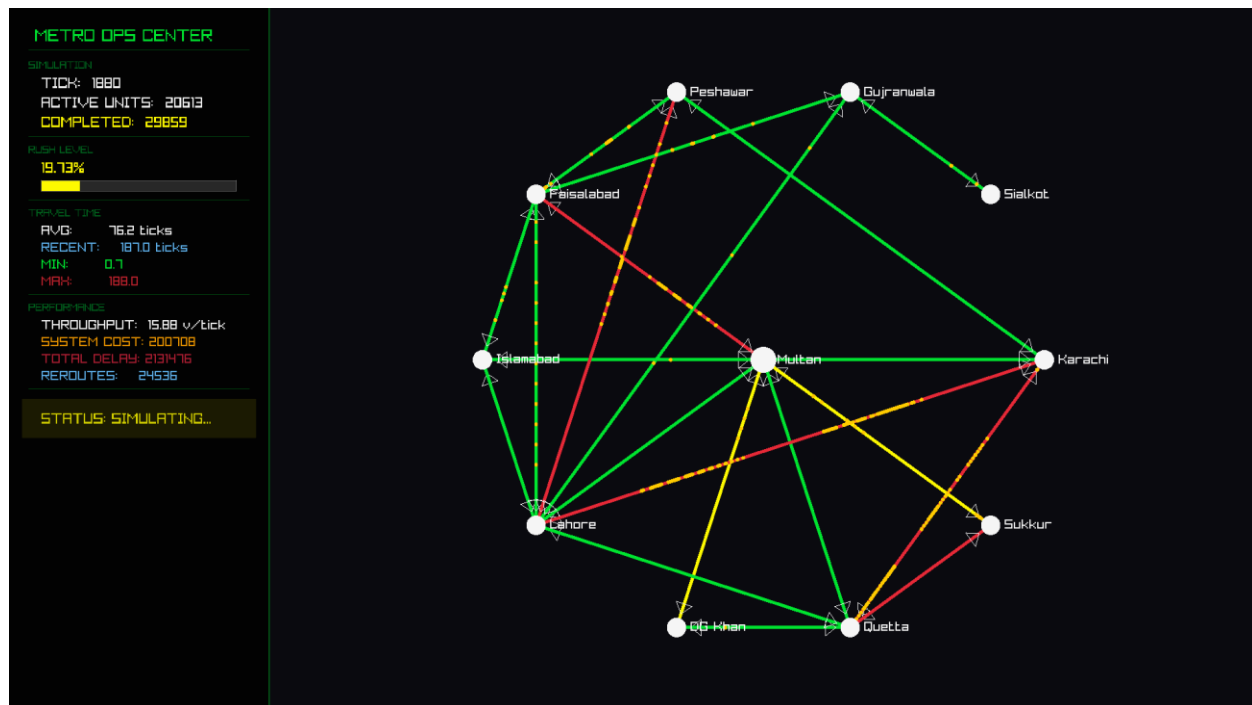
The graphical interface is built using the Raylib 5.0 library. The renderer draws each city as a labeled circle, each road as a directed arrow colored on a green-to-red congestion gradient ($p = 0 \rightarrow \text{green}$, $p = 1 \rightarrow \text{red}$). Vehicles appear as gold moving dots along their current road segment. A live statistics panel shows tick count, active vehicles, rush level bar, travel time stats (avg/windowed/min/max), throughput, system cost, total delay, and reroute count. Multan is positioned at the center as the network hub with spoke cities arranged radially.

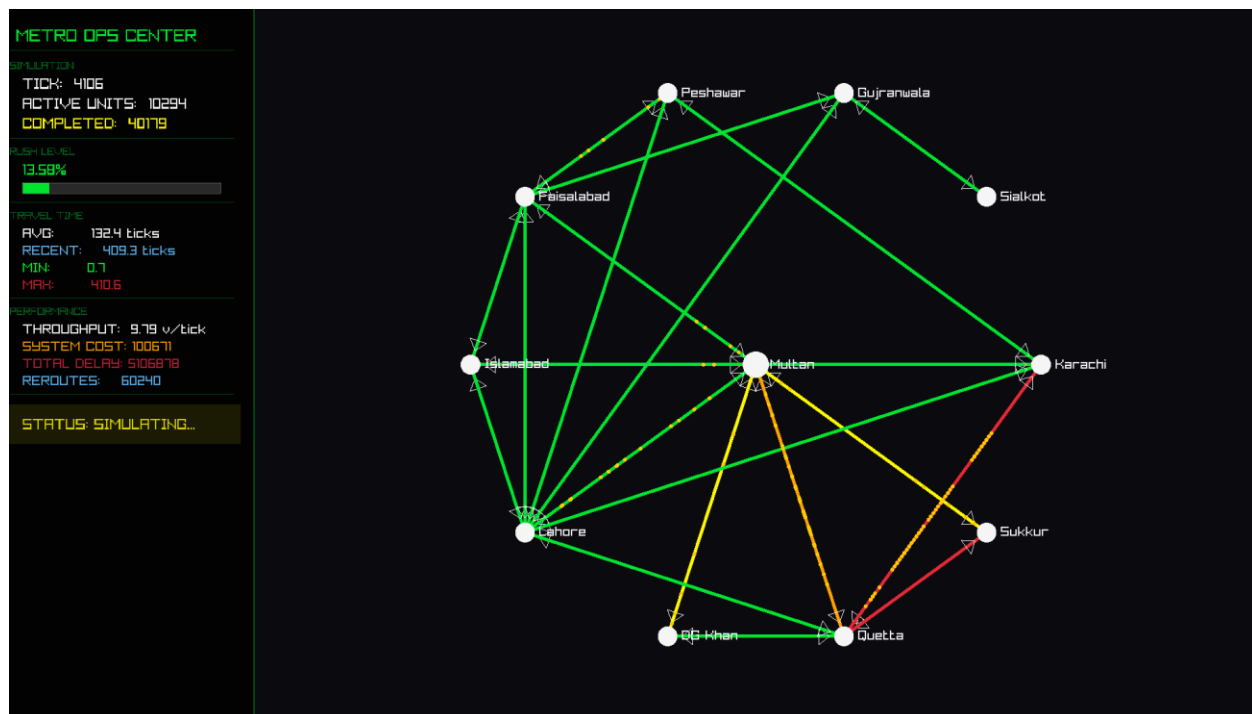
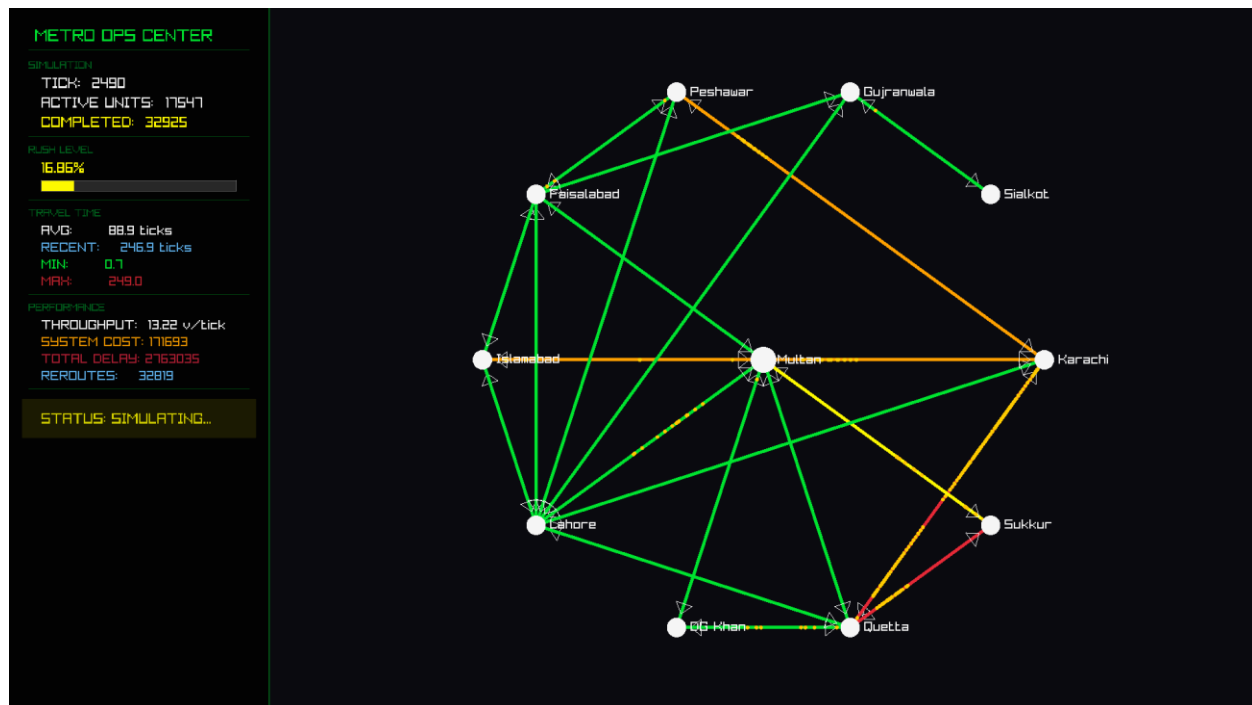


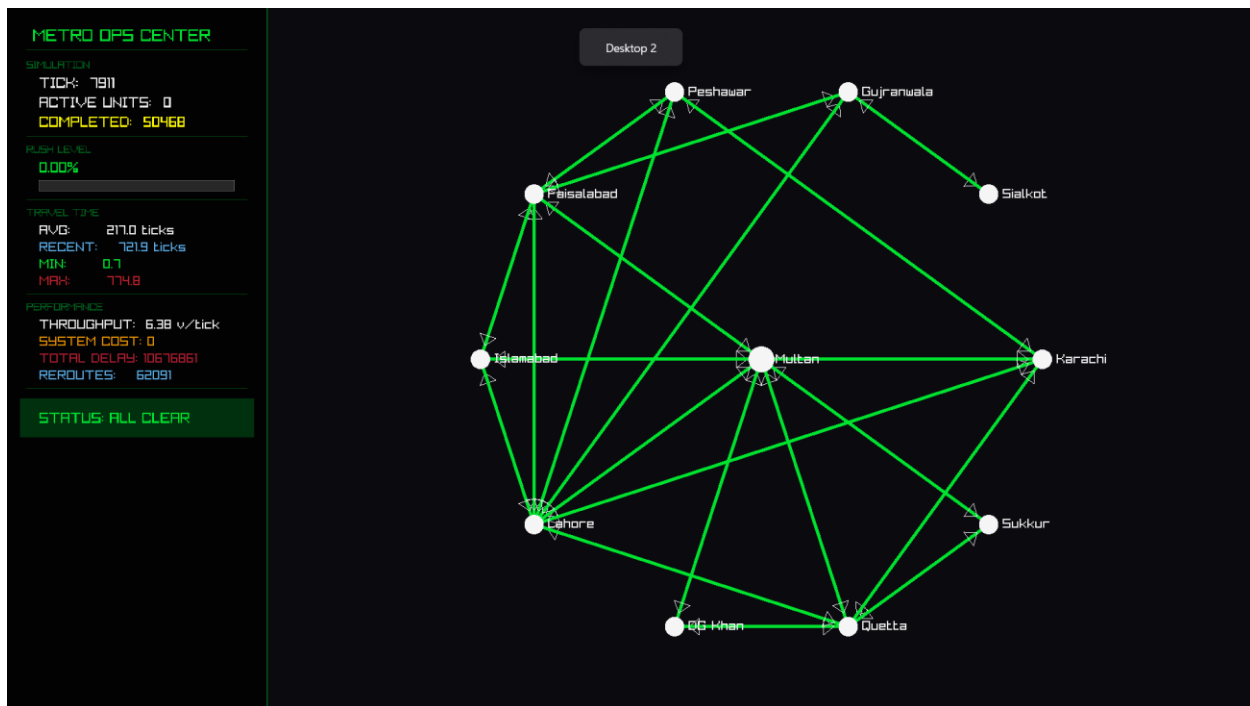
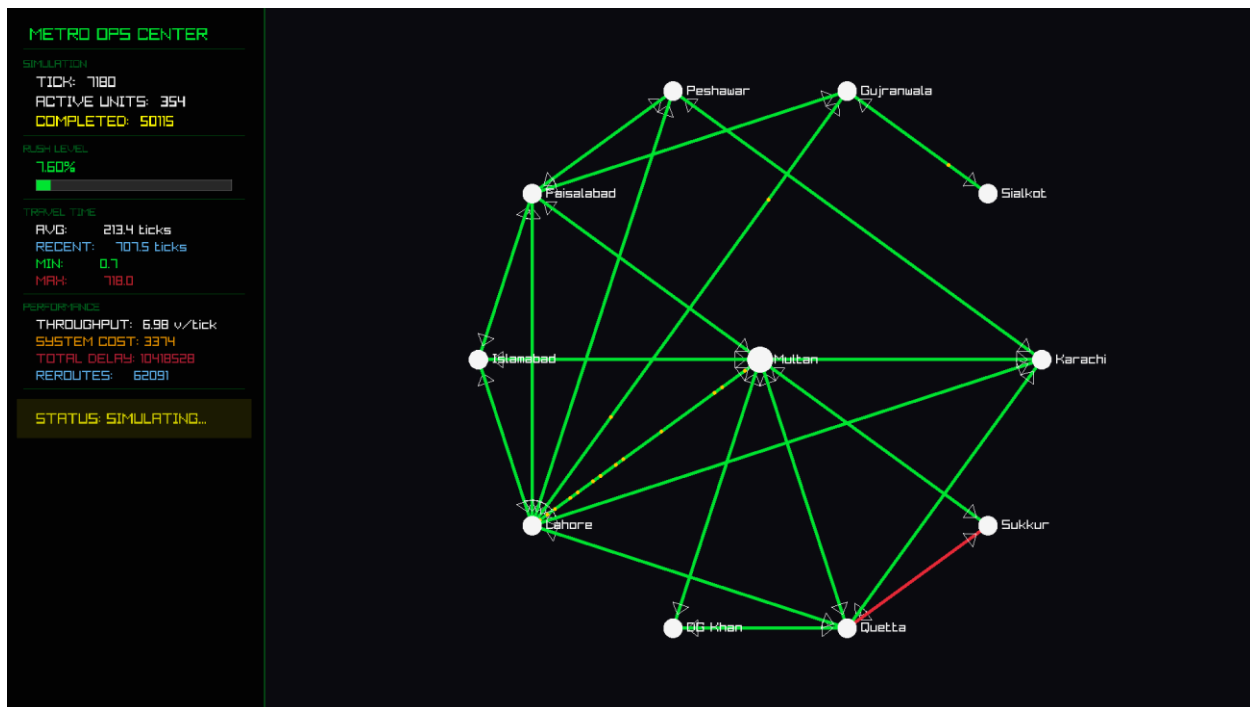












8.2 Performance Metrics Output

Metric	Formula	Description
Cumulative Avg Travel Time	$\text{totalTime} / \text{arrivedCount}$	Overall average across all completed trips

Metric	Formula	Description
Windowed Avg Travel Time	mean(last 100 arrivals)	Recent performance indicator
Min/Max Travel Time	tracked per arrival	Best and worst individual trips
P95 Travel Time	95th percentile of recent window	Tail latency — worst 5% experience
Throughput	arrivedCount / tickTime	Vehicles completed per tick
Total Delay	totalTime – totalFreeFlowTime	Excess time due to congestion
System Cost	$\Sigma(pa \times queue + pb \times congestion^2)$	Network-wide congestion cost
Average Rush Level	mean(currentVehicles/capacity)	Overall congestion ratio
Reroute Count	incremented on path change	How often vehicles switch routes

Output files: performance_metrics.txt, metrics.csv, car_timings.txt, map.txt

9. Time & Space Complexity Analysis

Notation: V = vertices (11), E = edges (~100), N = active vehicles (up to 50K), P = avg path hops (2–5), W = metrics window (100), T = ticks (up to 7000).

9.1 Per-Function Complexity

Operation	Time Complexity	Explanation
insertVertex()	O(1)	Array append
getIndex()	O(V)	Linear scan of vertex array
makeEdge()	O(1)	List push_back
DeleteVertex()	O(V·d + E)	Remove from all lists + shift + re-index
bfs() / dfs()	O(V + E)	Standard traversals
Dijkstra PQ (shortest_Path_bt看2_verification)	O((V + E) log V)	Min-heap extraction; lazy deletion of stale entries
minimumSpanningtree() (Prim's)	O(V ² ·d_max)	Linear scan — not yet upgraded to PQ

Operation	Time Complexity	Explanation
AverageRush() / total_System_Cost()	$O(V + E)$	Single pass over all edges
getEdgeDetails()	$O(V + d(v))$	getIndex + neighbor scan
All RoadDetails functions	$O(1)$	Arithmetic on scalar fields
All TrafficSignal functions	$O(1)$	Timer increments, comparisons
vehicle::getTakenPath()	$O(P)$	Iterates selected_path
addVehicle()	$O((V+E) \log V)$	Dijkstra for initial path
entrance() / entraingfromQueetoEdge()	$O(N \cdot V)$	Per-vehicle getIndex + edge lookup
ShortestPath()	$O(N \cdot (V+E) \log V)$	Dijkstra for every active vehicle
arrivalAtIntersection()	$O(N \cdot (V+E) \log V)$	Calls ShortestPath()
reached()	$O(N \cdot (V+P))$	Vehicle removal + metric recording
updateSignals()	$O(V^2 + V \cdot E)$	getEdges per vertex
recordArrival()	$O(P + W)$	Path iteration + window shift
printPerformanceMetrics() / printCSVRow()	$O(W \log W + E)$	P95 sort + system cost
injectSinusoidalTraffic()	$O(K \cdot (V+E) \log V)$	$K = 5\text{--}25$ vehicles per wave

9.2 Per-Tick Breakdown

One simulation tick costs:

updateSignals()	$\rightarrow O(V^2 + V \cdot E)$	
Vehicle time updates	$\rightarrow O(N)$	
reached()	$\rightarrow O(N \cdot V)$	
arrivalAtIntersection()	$\rightarrow O(N \cdot (V+E) \log V)$	← DOMINANT
entraingfromQueetoEdge()	$\rightarrow O(N \cdot V)$	
entrance()	$\rightarrow O(N \cdot V)$	
checkBrokenPath()	$\rightarrow O(N \cdot V)$	

Total per tick $\rightarrow O(N \cdot (V+E) \log V + V \cdot E)$

9.3 Overall Simulation Complexity

Metric	Complexity	Concrete ($V=11, E \approx 100$)
Per Dijkstra call	$O((V + E) \log V)$	~388 operations

Metric	Complexity	Concrete ($V=11$, $E \approx 100$)
Per simulation tick	$O(N \cdot (V+E) \log V + V \cdot E)$	$\sim 388N + 1,100$
Total simulation	$O(T \cdot N \cdot (V+E) \log V)$	$T \leq 7,000$
Startup injection	$O(C \cdot (V+E) \log V)$	$C = 50,000$
Total space	$O(V + E + N \cdot P + W)$	$\sim 250K$ entries

9.4 Scalability Analysis

The priority queue Dijkstra provides significant advantages as the graph grows:

V (cities)	V^2 (old linear scan)	$(V+E) \log V$ (current PQ)	Speedup
11	121	~ 388	—
50	2,500	$\sim 1,130$	2.2×
100	10,000	$\sim 2,660$	3.8×
500	250,000	$\sim 18,000$	13.9×
1,000	1,000,000	$\sim 30,000$	33×

9.5 Space Complexity Summary

Data Structure	Space	Purpose
<code>Gnode<t>[size]</code>	$O(V)$	Vertex storage
<code>list<weighted></code> per vertex	$O(E)$ total	Adjacency lists
<code>list<vehicle<t>></code>	$O(N)$	Active vehicle pool
<code>list<t></code> per vehicle (path)	$O(P)$ each, $O(N \cdot P)$ total	Vehicle paths
<code>priority_queue</code> (Dijkstra)	$O(E)$ worst case	Temporary per call
<code>vector<float></code> (window)	$O(W)$	Recent travel times

9.6 Summary

The dominant cost per tick is `ShortestPath()` which runs Dijkstra for every active vehicle at $O(N \cdot (V+E) \log V)$. For the current network with $V = 11$, $E \approx 100$, and N up to 50,000, a full simulation of $T = 7,000$ ticks completes efficiently. The priority queue optimization ensures the system is ready to scale to significantly larger networks — at $V = 1,000$ the PQ Dijkstra would be 33× faster than the previous linear-scan implementation.

10. Challenges and Solutions

10.1 Graph Construction and Directed Edge Representation

Representing a directed road network with rich per-edge metadata (BPR params, queue counts, signal state, discharge rate) required careful adjacency-list design. Solved by embedding RoadDetails structs with TrafficSignal objects directly into weighted edge entries in the adjacency list, enabling $O(1)$ access to all road properties during traversal.

10.2 Consistent Tick-Order Updates

Updating vehicle flows, queues, and signals in the wrong order caused vehicles to double-count or produce inconsistent states. Solved by enforcing a strict multi-phase tick order: signals → time decrement → arrivals → routing → queue transitions → entrance → cleanup → metrics.

10.3 Stale Entries in Dijkstra's Priority Queue

Dynamically changing edge weights (from BPR recalculation) produced stale priority-queue entries leading to suboptimal paths. Solved using lazy deletion: if (visited[nextNode]) continue skips already-settled nodes, allowing the heap to contain redundant entries without correctness issues.

10.4 Raylib Rendering Synchronization

Synchronizing the simulation tick rate with Raylib's frame rate required running the full simulation logic inside the Raylib draw loop with per-frame time decrements of 0.1 (vs. 2.0 in console mode), decoupling logical speed from visual frame rate via SetTargetFPS(60).

10.5 Scalable Vehicle Management

Managing 50,000+ vehicles with frequent insertions and removals required `std::list<vehicle>` instead of `std::vector` to enable $O(1)$ erase during iteration without iterator invalidation.

12. Entire Project Complexity — End-to-End Analysis

This section consolidates the complexity of the **entire simulation lifecycle** from startup to completion, providing a single holistic view of the system's computational profile.

12.1 Simulation Lifecycle Phases

The project executes in three distinct phases, each with its own complexity characteristics:

Phase	What Happens	Time Complexity	Concrete Cost (V=11, E≈100)
Phase 1: Setup	Insert 11 vertices, create ~100 directed edges	$O(V + E)$	~111 ops
Phase 2: Vehicle Injection	Create 50,000 vehicles, each gets a Dijkstra-computed initial path	$O(C \cdot (V+E) \log V)$	$50,000 \times 388 \approx$ 19.4M ops
Phase 3: Simulation Loop	Run up to 7,000 ticks; each tick reroutes all active vehicles, updates signals, processes queues	$O(T \cdot N \cdot (V+E) \log V)$	Up to $7,000 \times 50,000 \times 388 \approx$ 136B ops (theoretical max)

Note: N decreases each tick as vehicles arrive at their destinations, so the actual total is significantly lower than the theoretical maximum.

12.2 Complete Time Complexity Formula

The total time complexity of the entire project is:

$$T_{\text{total}} = T_{\text{setup}} + T_{\text{injection}} + T_{\text{simulation}}$$

$$\begin{aligned}
&= O(V + E) \\
&+ O(C \cdot (V + E) \log V) \\
&+ O\left(\sum_{t=1}^T [N(t) \cdot (V + E) \log V + V^2 + V \cdot E]\right)
\end{aligned}$$

Where $N(t)$ is the number of active vehicles at tick t (decreasing as vehicles arrive). The dominant term is the simulation loop, giving:

Overall: $O(T \cdot N_{\text{peak}} \cdot (V + E) \log V)$

12.3 Concrete Breakdown for This Project

Parameter	Value
V (vertices/cities)	11
E (directed edges/roads)	~100
C (initial vehicles injected)	50,000
N_{peak} (max active vehicles at any tick)	~50,000
T (max simulation ticks)	7,000
P (average path length)	2–5 hops

Parameter	Value
$(V + E) \log V$	≈ 388

Component	Operations	% of Total
Network setup	~111	< 0.001%
Initial Dijkstra routing (50K vehicles)	~19.4 million	~0.5%
Simulation loop (signals + routing + movement)	~3.7 billion (estimated with decay)	~99.5%
Metrics I/O (70 reports $\times$ $O(W \log W)$)	~50,000	< 0.001%

12.4 Complete Space Complexity

The total memory footprint of the entire project:

$$S_{\text{total}} = S_{\text{graph}} + S_{\text{vehicles}} + S_{\text{dijkstra}} + S_{\text{metrics}}$$

$$\begin{aligned}
 &= O(V + E) && // \text{Graph: 11 nodes + ~100 edges with RoadDetails} \\
 &+ O(N \cdot P) && // \text{Vehicles: 50K vehicles } \times \text{ path lists + selected} \\
 &\text{_paths} && \\
 &+ O(V + E) && // \text{Dijkstra: temporary arrays + priority queue (per call)} \\
 &+ O(W) && // \text{Metrics: sliding window of 100 recent travel times}
 \end{aligned}$$

Overall Space: $O(N \cdot P + V + E) \approx O(N \cdot P)$ for large N

Data	Estimated Memory
Graph (11 nodes, ~100 edges $\times$ RoadDetails)	~50 KB
50,000 vehicles $\times$ (path + selected_path + state)	~12 MB
Dijkstra temporaries (per call, reused)	~4 KB
Metrics window (100 floats)	< 1 KB
Raylib rendering state	~2 MB
Total estimated	~15 MB

12.5 Complexity Class of the Entire System

Aspect	Complexity Class
Time (total)	$O(T \cdot N \cdot (V + E) \log V)$ — Polynomial; dominated by per-tick Dijkstra rerouting for all vehicles

Aspect	Complexity Class
Space (total)	$O(N \cdot P + V + E)$ — Linear in the number of vehicles; path storage dominates
Per-tick time	$O(N \cdot (V + E) \log V + V \cdot E)$ — Linear in N with log-factor from the priority queue
Startup time	$O(C \cdot (V + E) \log V)$ — Linear in vehicle count; one Dijkstra per vehicle

The system is **polynomial-time** and **linear-space**, making it computationally tractable for real-time simulation on consumer hardware. The priority queue Dijkstra ensures the system scales gracefully — doubling V does not quadruple cost (as it would with linear-scan Dijkstra), but instead increases cost by roughly $2 \times \log$ factor.

13. Conclusion

The Adaptive Traffic Flow Optimization System successfully demonstrates the practical application of core Data Structures concepts — directed graphs, priority queues, linked lists, and queue-based flow models — to a real-world Complex Engineering Problem. The system integrates six mathematically-grounded components (graph model, BPR travel time, Dijkstra PQ routing, queue dynamics, adaptive signal control, and performance metrics) into a cohesive, modular C++ simulation that executes in real time with Raylib visualization.

The Dijkstra implementation was upgraded from a linear-scan $O(V^2)$ approach to a priority-queue $O((V + E) \log V)$ implementation, providing up to $33 \times$ speedup for larger networks. The implementation fulfills all CEP assessment dimensions: the graph and Dijkstra data structures are correctly designed and efficiently implemented; the BPR and queue models faithfully replicate the mathematical specification; the code is organized for reusability with clean class boundaries; and the system handles edge cases including zero-capacity roads, isolated intersections, and identical source-destination pairs.

References

1. CEP Problem Specification: Traffic Flow Optimization System Using Graph Theory. EC-200 Data Structures, NUST College of E&ME, 2026.
2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
3. Bureau of Public Roads. (1964). *Traffic Assignment Manual*. U.S. Department of Commerce, Urban Planning Division.
4. Raylib Documentation. (2024). A simple and easy-to-use library to enjoy videogames programming. <https://www.raylib.com>

5. Dijkstra, E. W. (1959). A note on two problems in connexion with graphs.
Numerische Mathematik, 1(1), 269–271.